Challenge 2 — Operazioni Critiche in GET (CSRF)

Le rotte di cambio ruolo (`set-admin`, `set-revisor`, `set-writer`) erano implementate come **GET**, bypassando la protezione CSRF di Laravel. Un attacco CSRF ha promosso Kevin Ross (id 7, nessun privilegio) ad admin con la sola collaborazione inconsapevole di un admin reale loggato. Mitigato convertendo le tre rotte a **PATCH**, sostituendo i link con form `@csrf` / `@method('PATCH')`, e rinforzando i cookie di sessione.

Scenario e vulnerabilità

```
GET|HEAD  admin/{user}/set-admin    → AdminController@setAdmin
GET|HEAD  admin/{user}/set-revisor  → AdminController@setRevisor
GET|HEAD  admin/{user}/set-writer   → AdminController@setWriter
```

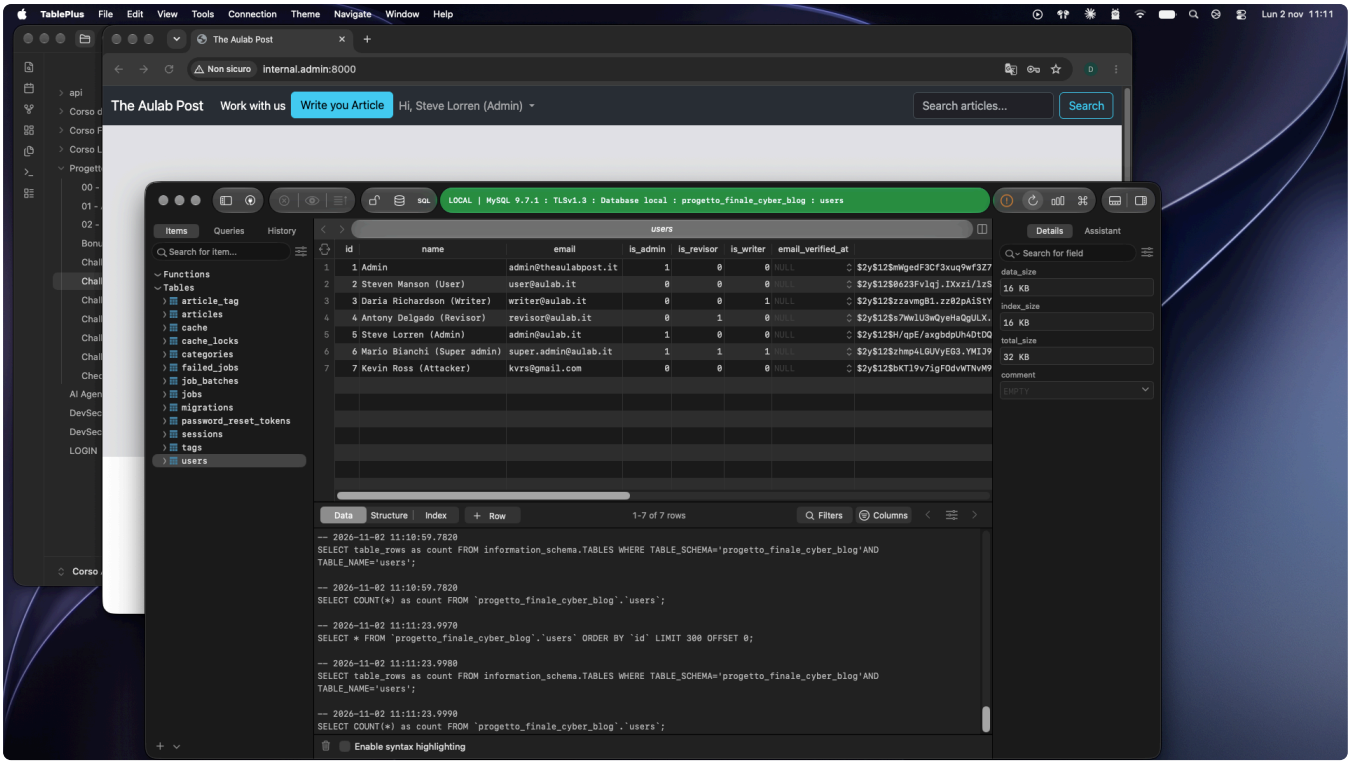
In Laravel il token `@csrf` protegge solo le richieste con metodi "non sicuri" (POST/PUT/PATCH/DELETE). Una rotta GET non lo richiede, quindi è invocabile da qualunque origine — un link, un'immagine, uno script esterno — sfruttando i cookie di sessione già presenti nel browser della vittima.

Attacco

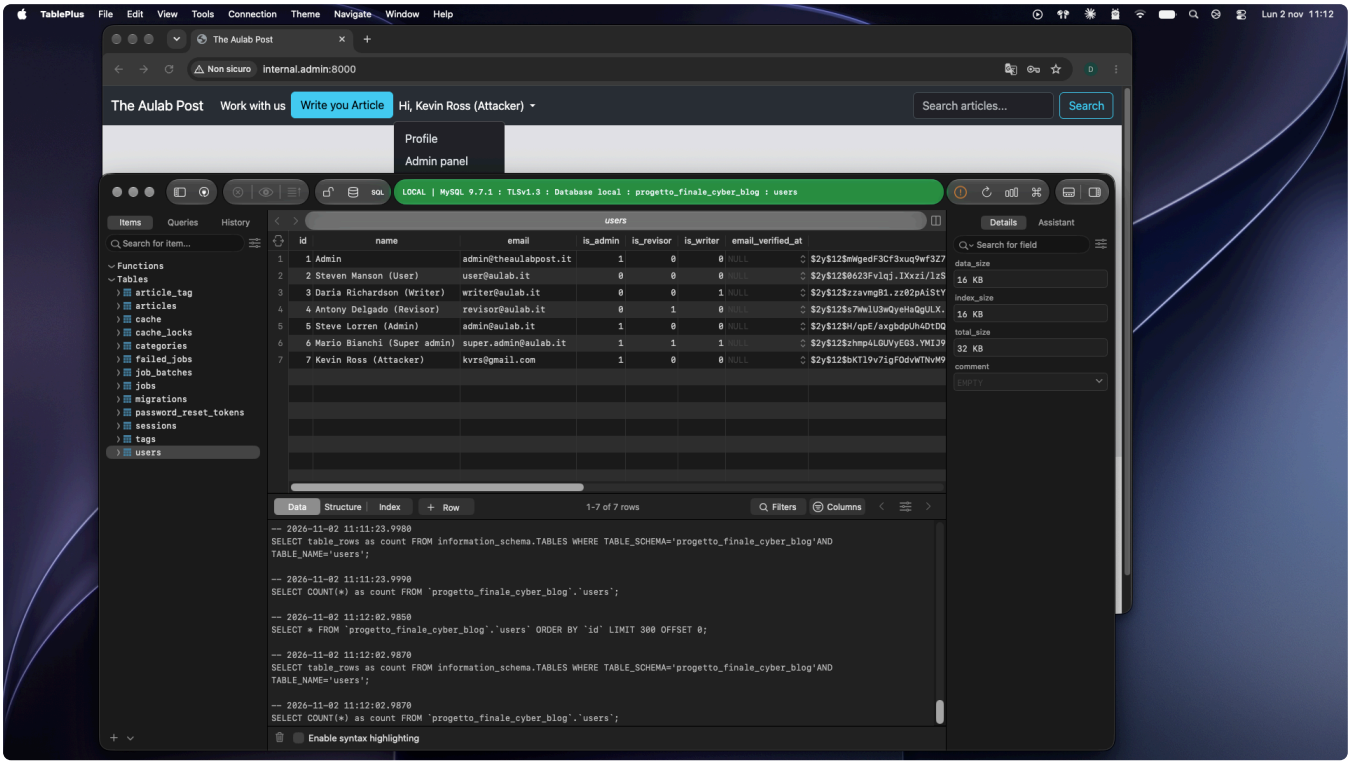
Il link malevolo è un elemento `<a>` reale, cliccato programmaticamente dopo 5 secondi mentre la vittima è distratta da una pagina-pretesto:

È una **navigazione di primo livello con metodo GET** — esattamente il caso che `SameSite=Lax` (default in molte configurazioni) lascia passare: un link cliccato porta con sé i cookie di sessione anche cross-origin, cosa che un `fetch()` o un `<img src="...">` non avrebbero fatto.

Setup del test: login come admin reale (vittima, es. Steve Lorren) → apertura della pagina-trappola nello stesso browser, puntata su `admin/7/set-admin` (Kevin Ross, is_admin = 0) → attesa 5 secondi → verifica nel DB.



Dopo il click automatico per conto della vittima, Kevin Ross risulta is_admin = 1; la sua stessa sessione mostra ora "Hi, Kevin Ross (Attacker)" con accesso all'Admin panel.



Mitigazione

Defense in Depth su tre strati indipendenti.

1 — Rotte da GET a PATCH, coerenti con le altre azioni del controller (già POST/PUT/DELETE):

2 — Link sostituiti con form protetti da CSRF in `components/requests-table.blade.php`:

3 — Cookie di sessione rinforzati nel `.env`:

Con `SameSite=Strict` l'attacco sarebbe già neutralizzato (il cookie non parte mai su richieste cross-site), ma la mitigazione 1–2 resta necessaria: un domani qualcuno potrebbe abbassare quel valore nel `.env` senza conoscerne la motivazione, o un browser datato potrebbe non rispettarlo correttamente. Avere anche le rotte sul verbo corretto rende l'applicazione sicura anche se la configurazione dei cookie dovesse regredire.

Nota d'ambiente: `SESSION_SECURE_COOKIE=true` richiede HTTPS. In locale (HTTP semplice) va impostato `false` per non invalidare la sessione; in produzione, dietro HTTPS reale, va mantenuto `true`.

Verifica

Rilanciando la pagina-trappola, il browser esegue comunque una GET verso la rotta — ma questa ora accetta solo PATCH. Il server risponde 405, e il ruolo non cambia:

